docs/handover.md - webRagSys 進捗管理

本ファイルは進捗管理用mdである。フェーズ完了ごとに更新する。形式：方針（計画）→実装（実行）→結果→残タスク→実行コマンド。人間とAIの双方がコンテキスト0%から再開できる粒度で相当詳しく記録する。

2026-09-19 初期化フェーズ（README＋進捗管理先行）

方針（計画）

- 対象フォルダ：webRagSys（親フォルダの兄弟フォルダ。他フォルダは読込不要）
- 要件定義書（20章）を受領済み。目的は社内文書CRUD＋RAG＋AIチャット。
- 学習目的の本質：Python → API → DB → LLM → RAG。
- 技術選定の確定事項：
  - LLM／Embedding：OpenAI利用（本番）。既定は課金回避の偽実装
  - RAGフレームワーク：LangChain固定
  - VectorDB：pgvector（PostgreSQL拡張で一元管理）
  - DB方針：PostgreSQL単一
  - 対象者：Python中級者、RAG初心者向けにbeginner_overview.mdを作成
- 順序の指定：
  - docs（beginner_overview等）とdiagramsはテストの後に作成し、PDF生成する
  - READMEと進捗管理用mdは先行作成する
- AGENTS.mdは親フォルダのマスタAGENTS.mdを継承し、内容に従う
- 参考資料：
  - stockChecker/docs/beginner_overview.md（構成見本）
  - stockChecker/diagrams（01_flowchart、02_sequence、03a、03b、04_mindmap、05_state、.mmd＋.md＋.pdf、convert_to_pdf.ps1＋puppeteer設定）
  - stockChecker/README.md（README構成見本）
  - webScraping/docs/handover.md（進捗管理の記録粒度見本）
- PDF方針：
  - 図PDFはMermaid CLI（npx @mermaid-js/mermaid-cli、--pdfFit、03aのみ横A4）で生成
  - docs PDFはMarkdown→PDFで生成（手段は実装確定後に再確認）
- セキュリティ：ローカルパス、APIキー、個人ID、認証情報は一切記録しない。機密情報は.env（Git除外）のみ。

実装（実行）

- webRagSysフォルダの空を確認（Get-ChildItemで0件）
- 環境確認：Python 3.11.15、pip 26.1.2、git 2.54.0、Docker未検出（CommandNotFound）
- webRagSys/docsフォルダ作成
- webRagSys/AGENTS.md作成：
  - 親マスタ02テンプレート継承（概要、環境、禁止情報、GitHub運用、分割コミット、handover必須更新）
  - webRagSys固有化（FastAPI、PostgreSQL＋pgvector、LangChain、偽実装既定、ディレクトリ構造、RAG注意点）
  - 規約：先頭コメント7項目、config集約、main／validate／execute／format分離、単一責務、型ヒント必須、PEP8、例外明示
  - docs／diagramsはテスト後＋PDF化、README／handoverは先行＋随時更新を明記
- webRagSys/README.md雛形作成（要件定義書14章準拠）：
  - 概要、機能、技術表、構成ツリー、セットアップ、起動、API一覧6件、RAG概要図、使用例curl、テスト、ドキュメント案内、拡張案、完成条件、注意点
  - .env.example運用、偽実装既定を明記
- 本handover.md作成（本ファイル）

結果

- 3ファイル生成完了：
  - webRagSys/AGENTS.md
  - webRagSys/README.md
  - webRagSys/docs/handover.md
- 単体テスト：未実施（実装なしのため）

- Lint：未実施（実装なしのため）
- PDF：未生成（テスト後に実施する指定のため）
- 発見事項：
  - Docker未導入のため、完成条件「Dockerで環境構築」は後工程でブロッカーになる可能性あり
  - Python実測3.11のため、AGENTS.mdでは3.10+互換として記載
  - RUG表記はRAGとして扱う

残タスク

- [ ] 骨格実装：config.py、requirements.txt、.env.example、.gitignore、Dockerfile、docker-compose.yml、src雛形
- [ ] 文書CRUD実装：schemas、models、repository、routers/documents
- [ ] RAG実装：chunker、embeddings（本番／Fake切替）、vectorstore（pgvector）、chain（LangChain）
- [ ] chat実装：routers/chat（検索→プロンプト組立→LLM呼出→回答＋参照ID）
- [ ] LLM偽実装：provider切替（USE_FAKE既定true）
- [ ] テスト実装＋実行：pytest全件成功、ruffエラー0（docs着手条件）
- [ ] docs本格作成（テスト後）：beginner_overview.md（Python中級・RAG初心者向け、LangChain中心）、README追記
- [ ] diagrams作成（テスト後）：01_flowchart、02_sequence、03a_class_rag、03b_class_modules、04_mindmap、05_state（.mmd＋.md）
- [ ] PDF生成：docs PDF、diagrams PDF（Mermaid CLI＋puppeteer設定複製）
- [ ] Docker検証：PostgreSQL＋pgvector起動、API接続
- [ ] GitHub公開：public、分割コミット、push前禁止情報チェック

実行コマンド

```
# 環境確認（実施済み）
python --version
pip --version
git --version

# フォルダ確認（実施済み）
Get-ChildItem -Force -LiteralPath "webRagSys"

# 今後のテスト／Lint（予定）
python -m pytest tests -q
python -m ruff check src tests config.py

# 今後の起動（予定）
uvicorn src.api.main:app --reload

# 今後の図PDF変換（予定、Node.js前提）
# npx -y @mermaid-js/mermaid-cli -i diagrams/01_flowchart.mmd -o diagrams/01_flowchart.pdf --pdfFit
```

実行成果物

- webRagSys/AGENTS.md
- webRagSys/README.md
- webRagSys/docs/handover.md

公開禁止情報

公開禁止情報は含まれていません

2026-09-19 実装フェーズ（骨格＋CRUD＋RAG＋chat＋テスト）

方針（計画）

- 先行作成済みのAGENTS.md、README.md雛形に従い、最小構成で完成条件1〜8と11を満たす

- LangChain固定、既定USE_FAKE=trueで課金なし。EmbeddingはFakeベクトル、LLMはFake定型文
- DBは本番PostgreSQL＋pgvector、テストとドライバ欠落時はSQLite退避で動作保証する
- docsとdiagramsはテストの後に作成する指定のため、本フェーズでは作らない

実装（実行）

- 骨格：requirements.txt、config.py（Settings集約）、.env.example、.gitignore、Dockerfile、docker-compose.yml（pgvector/pg16）、pyproject.toml（ruff設定）
- DB：src/db/database.py（engine遅延生成、Postgres拡張、get_db、init_db）、src/db/models.py（documents、document_chunks、Postgres時Vector／他時JSON）
- スキーマ：src/schemas/document.py、src/schemas/chat.py
- RAG：src/rag/chunker.py（LangChain優先＋単純分割退避）、src/rag/embeddings.py（Fake既定／OpenAI本番）、src/rag/vectorstore.py（save／delete／cosine検索）、src/rag/chain.py（answer_question）
- LLM：src/llm/fake_provider.py、src/llm/provider.py（USE_FAKE切替）
- API：src/api/main.py（create_app、health、startup）、src/api/routers/documents.py（CRUD5件、404／422処理、登録更新時にchunks再生成）、src/api/routers/chat.py（POST /chat、500処理）
- テスト：tests/conftest.py（SQLite＋dependency override）、tests/test_documents.py（5件）、tests/test_chat.py（RAG利用＋chunker）
- 修正：
  - 初回pytestでpsycopg2欠落エラー→database.pyを遅延生成＋SQLite退避に修正
  - ruffでUP009／S101／B008／E501等→pyprojectでUP009とB008除外、testsのS101除外、行長100、E501の3か所改行、main.pyのbroad-exceptをOSError／RuntimeError＋loggingに修正

結果

- 単体テスト：7件すべて成功（python -m pytest tests -q）
- Lint：ruffエラー0（python -m ruff check src tests config.py → All checks passed）
- 検証：POST /documents→GET／PUT／DELETE、存在しないIDで404、POST /chatでFAKE回答＋source_ids取得を確認
- PDF：未生成（テスト後工程のため）

残タスク

- [ ] docs本格作成（テスト後）：beginner_overview.md（Python中級・RAG初心者向け）、README追記
- [ ] diagrams作成（テスト後）：01_flowchart、02_sequence、03a_class_rag、03b_class_modules、04_mindmap、05_state（.mmd＋.md）
- [ ] PDF生成：docs PDF、diagrams PDF（Mermaid CLI＋puppeteer設定）
- [ ] Docker検証：Docker導入後にcompose起動とPostgreSQL＋pgvector接続確認
- [ ] GitHub公開：public、分割コミット、push前禁止情報チェック

実行コマンド

```
# 依存
pip install -r requirements.txt
# テスト／Lint
python -m pytest tests -q
python -m ruff check src tests config.py
# 起動
uvicorn src.api.main:app --reload
```

実行成果物

- config.py、requirements.txt、pyproject.toml、Dockerfile、docker-compose.yml、.env.example、.gitignore
- src/api／schemas／db／rag／llm／utils一式
- tests/conftest.py、test_documents.py、test_chat.py

公開禁止情報

公開禁止情報は含まれていません

2026-09-19 ドキュメント・図フェーズ（テスト後）

方針（計画）

- 指定順序どおりテストの後にdocsとdiagramsを作成し、両方PDF化する
- beginner_overview.mdはstockChecker構成を踏襲し、対象をPython中級・RAG初心者に切替える
- RAGフレームワークはLangChain固定、偽実装で課金を避ける
- 図はstockCheckerと同形式の6種（.mmd＋.md＋.pdf）

実装（実行）

- docs/beginner_overview.md作成（実コード照合済み）：
  - 1章：構成と役割表、2章：API→RAG→DB／LLM依存図、3章：登録→RAG化→質問応答フロー
  - 4章：Depends、from_attributes、TextSplitter、Fake切替、pgvector、コサイン、.env区別
  - 付録：STEP1-12学習順
- diagrams 6種作成（.mmd＋.md）：
  - 01_flowchart：起動→CRUD分岐→RAG化→chat分岐
  - 02_sequence：登録と質問応答の通信
  - 03a_class_rag：Chunker／Embeddings／VectorStore／Chain／LLM
  - 03b_class_modules：Main／Routers／DB／Settings
  - 04_mindmap：全体概念階層
  - 05_state：PENDING→CHUNKING→EMBEDDING→SAVING→COMPLETED
- diagrams/convert_to_pdf.ps1＋puppeteer設定2件をstockCheckerから流用（03aのみ横A4）
- PDF生成：図6件はmermaid-cli、docs 3件（beginner_overview、handover、README）はmd-to-pdf
- README追記：ドキュメント節を本格版に更新
- 不具合と修正：
  - convert_to_pdf.ps1がPS5.1で解析エラー→原因は矢印文字のUTF-8 BOMなし問題、英字表記に書換えて解決、直接実行で6件変換を確認
  - md-to-pdfのpdf-options指定がPowerShell引用で失敗→既定オプションで生成に変更
  - pytest間欠失敗（no such table: documents）→原因はdb fixture実行時点でmodels未importのためcreate_allが空振りし、旧test.db残存時のみ偶然成功していた。conftestにmodels import追加＋tmp_path毎テスト分離で解決、3連続7件成功を確認

結果

- 単体テスト：7件すべて成功（3連続確認）
- Lint：ruffエラー0
- PDF：図6件＋docs 3件の計9件を生成
- 図PDF：01_flowchart、02_sequence、03a_class_rag、03b_class_modules、04_mindmap、05_state
- docs PDF：beginner_overview.pdf、handover.pdf、README.pdf

残タスク

- [ ] Docker検証：Docker導入後にcompose起動とPostgreSQL＋pgvector接続確認
- [x] GitHub公開：public、分割コミット、push前禁止情報チェック

実行コマンド

```
# テスト／Lint
python -m pytest tests -q
python -m ruff check src tests config.py
# 図PDF再生成
./diagrams/convert_to_pdf.ps1
# docs PDF再生成
npx md-to-pdf docs/beginner_overview.md
npx md-to-pdf docs/handover.md
npx md-to-pdf README.md
# 起動
uvicorn src.api.main:app --reload
```

実行成果物

- docs/beginner_overview.md＋.pdf
- docs/handover.md＋.pdf（本ファイル）
- README.md＋README.pdf（追記済み）
- diagrams 6種（.mmd＋.md＋.pdf）、convert_to_pdf.ps1、puppeteer設定2件

公開禁止情報

公開禁止情報は含まれていません

2026-09-19 GitHub公開フェーズ

方針（計画）

- publicリポジトリで公開し、履歴が読める分割コミットにする
- push前に公開禁止情報（.env、実パス、キー）をチェックする
- Dockerは未導入のため検証は保留し、導入後の手順を残す

実装（実行）

- 状態確認：Docker未検出、gh 2.96.0認証済み（HTTPS）、git未初期化を確認
- git init -b main、user.nameとuser.emailをリポジトリローカルに設定（noreply形式）
- 分割コミット6件：
  - 骨格・設定・Docker基盤
  - DB・スキーマ・RAG・LLM偽実装
  - API（documents CRUD・chat）
  - テスト（CRUD・chat偽実装）
  - docs・図・PDF
  - srcパッケージ初期化ファイル
- push前チェック：.env未追跡、実パスなし、キーなしを確認
- gh repo create webRagSys --public --source . --remote originで作成
- git push -u origin mainで公開

結果

- 公開先：GitHub上のwebRagSysリポジトリ
- mainがorigin/mainを追跡する状態を確認
- git statusはクリーン（handover更新分を除く）

残タスク

- [ ] Docker検証：Docker Desktop導入後にdocker compose up --buildとPostgreSQL＋pgvector接続確認
- [ ] 本フェーズのhandover更新分をコミット＋pushする

実行コマンド

```
git log --oneline
git push -u origin main
```

公開禁止情報

公開禁止情報は含まれていません

2026-09-19 起動検証フェーズ

方針（計画）

- 完成条件のFastAPI起動、CRUD動作、RAG回答を実HTTPで検証する
- Docker未導入のためSQLite退避起動で検証し、Postgres検証は保留する

実装（実行）

- 不具合：uvicorn起動直後に失敗、原因はinit_dbがURL文字列でPostgres判定しSQLite退避時もCREATE EXTENSIONを実行していた
- 修正：src/db/database.pyのinit_dbを実engine方言（engine.dialect.name）判定に変更
- 再検証：pytest 7件成功、ruffエラー0を確認後に実HTTP検証を実施
- 実HTTP検証（port 8001、同一セッションで起動→検証→停止）：
  - GET /health → ok
  - POST /documents → id=1で作成
  - GET /documents → 1件
  - GET /documents/1 → 詳細取得
  - POST /chat → FAKE回答＋source_ids=1
  - GET /documents/99999 → 404

結果

- 起動検証：すべて成功（日本語表示乱れはコンソール出力の文字化けのみでアプリは正常）
- テスト：7件成功、ruffエラー0を再確認

残タスク

- なし（Docker検証は対象外化、以下フェーズ参照）

実行コマンド

```
python -m pytest tests -q
python -m ruff check src tests config.py
python -m uvicorn src.api.main:app --reload
```

公開禁止情報

公開禁止情報は含まれていません

2026-09-19 Docker対象外化フェーズ

方針（計画）

- storage制約のためDocker検証を計画から外す（2026-09-19決定）
- Dockerfileとdocker-compose.ymlは構成見本として残し、削除しない
- 公式最少約6GB、本件規模で2〜4GB程度の見込みを確認済み
- 正規の動作確認手段はローカル起動（uvicorn＋SQLite退避／PostgreSQL直結）とする

実装（実行）

- AGENTS.md注意点にDocker検証対象外を追記
- READMEの起動方法と完成条件を更新（Docker項目を対象外化）
- 本handoverの残タスクを更新

結果

- テスト、Lintは次項で再確認する

残タスク

- なし

実行コマンド

```
python -m pytest tests -q
python -m ruff check src tests config.py
```

公開禁止情報

公開禁止情報は含まれていません

2026-09-20 docs図埋め込みフェーズ（beginner_overview PDFの図表示修正）

方針（計画）

- docs PDF内でMermaid図が生コード表示になる不具合を修正する
- 原因は変換道具の差：diagramsはmermaid-cli（絵対応）、docsはmd-to-pdf（mermaid非対応）
- .mmd不足ではない（diagramsに6件存在を確認済み）
- 画像埋め込み方式を採用：mermaid部分をPNG化してmdに貼り、md-to-pdfでPDF化する
- vscode-pdf NextはPDF閲覧専用のため不採用（md変換不可）

実装（実行）

- docs/assetsフォルダ作成、beginner_overview.md内4ブロックをmmd分離：
- overview_modules.mmd（2.1 依存関係図）、overview_flow.mmd（3.1 全体フロー）
- overview_rag.mmd（3.3 RAG化）、overview_chat.mmd（3.4 質問応答）
- mermaid-cliで4件PNG生成（日本語表示を確認済み）
- beginner_overview.mdの各mermaidブロックを画像参照＋details（元コード保持）に置換
- md-to-pdfでbeginner_overview.pdf再生成
- handover.mdはmermaidなしのため対象外、README.mdも対象外

結果

- 単体テスト：7件すべて成功（python -m pytest tests -q）
- Lint：ruffエラー0
- PDF：画像5件埋め込み確認（pypdfで検証）、425KB→527KB
- 図PDF：diagrams 6件は変更なし

残タスク

- なし

実行コマンド

```
# PNG生成
npx.cmd -y @mermaid-js/mermaid-cli -i docs/assets/overview_modules.mmd -o docs/assets/overview_modules.png
npx.cmd -y @mermaid-js/mermaid-cli -i docs/assets/overview_flow.mmd -o docs/assets/overview_flow.png
npx.cmd -y @mermaid-js/mermaid-cli -i docs/assets/overview_rag.mmd -o docs/assets/overview_rag.png
npx.cmd -y @mermaid-js/mermaid-cli -i docs/assets/overview_chat.mmd -o docs/assets/overview_chat.png
# docs PDF再生成
npx md-to-pdf docs/beginner_overview.md
# テスト／Lint
python -m pytest tests -q
python -m ruff check src tests config.py
```

実行成果物

- docs/assets/overview_*.mmd＋.png（各4件）
- docs/beginner_overview.md＋.pdf（画像埋め込み版）
- docs/handover.md（本ファイル）

公開禁止情報

公開禁止情報は含まれていません

2026-09-20 venv標準化フェーズ（.venv運用の文書化）

方針（計画）

- 仮想環境はvenv標準に統一する（他案件のconda envを流用しない）
- 配置はプロジェクト直下の.venvとする
- .venvは絶対パスを含むため.gitignoreでGit除外する（親AGENTS.mdにも同旨を追記）
- 手順はcd移動→venv作成→Activate→pip導入→.env複製→uvicorn起動の順に固定する

実装（実行）

- 親AGENTS.mdの開発環境（前提）節と02テンプレート節にvenv標準3行を追記
- webRagSys/.gitignoreに.venv/とvenv/を追記（activate系の絶対パス混入防止）
- webRagSys/README.mdのセットアップ節と起動節をvenv手順に更新（実パスは記載なし）
- webRagSys/AGENTS.mdの開発環境節とビルド節をvenv手順に更新
- .venvは利用者側で作成済みのため、本作業では文書更新のみ行う

結果

- テスト、Lintは次項で再確認する

残タスク

- なし

実行コマンド

```
cd <プロジェクト直下>
python -m venv .venv
.¥.venv¥Scripts¥Activate.ps1
pip install -r requirements.txt
copy .env.example .env
uvicorn src.api.main:app --reload
```

実行成果物

- 親AGENTS.md（venv標準追記）
- webRagSys/.gitignore、README.md、AGENTS.md、docs/handover.md（本ファイル）

公開禁止情報

公開禁止情報は含まれていません

2026-09-20 SQLite運用フェーズ（.env修正と起動検証）

方針（計画）

- DockerなしではPostgreSQLサーバが存在しないためSQLite退避を正規運用とする
- 起動失敗の原因はDATABASE_URLがdocker専用host名dbを指していたこと
- psycopg2導入済みのためSQLite退避分岐に入らずPostgres直行していた
- 既存.venvは3.9.7だが本検証では動作したため版更新は保留する

実装（実行）

- .envのDATABASE_URLを `sqlite:///./app.db` に変更（1行のみ）
- 実HTTP検証（port 8001、起動→検証→停止）：
- GET /health → ok
- POST /documents → id=1で作成
- GET /documents → 1件
- GET /documents/1 → 詳細取得
- POST /chat → FAKE回答＋source_ids=1
- 検証後にapp.dbを削除（実行生成物のため）

結果

- 起動検証：すべて成功（日本語表示乱れはコンソール出力の文字化けのみでアプリは正常）
- テスト：7件成功（.venvのPython 3.9.7で実行）、ruffエラー0

残タスク

- [ ] .venvをPython 3.11で作り直す（現3.9.7、案件要求3.10互換のため）

実行コマンド

```
.¥.venv¥Scripts¥Activate.ps1
python -m pytest tests -q
python -m ruff check src tests config.py
python -m uvicorn src.api.main:app --port 8001
```

実行成果物

- .env（DATABASE_URLをSQLiteに変更、Git対象外のため差分なし）
- docs/handover.md（本ファイル）

公開禁止情報

公開禁止情報は含まれていません

2026-09-20 第5章追記フェーズ（操作と内部の対応）

方針（計画）

- /docsの4分類と群別Code対応表と内部traceを初心者文書に集約する
- 新規文書は作らずbeginner_overview.mdへ第5章追記する
- 図は現行方式（mermaid PNG化＋details併記）を踏襲する

実装（実行）

- docs/assets/overview_ops.mmd＋.png（操作Code対応図）
- docs/assets/overview_internals.mmd＋.png（内部処理図）
- beginner_overview.mdへ第5章（5.1分類、5.2対応表、5.3内部trace、5.4見分け方）追記
- md-to-pdfでbeginner_overview.pdf再生成
- READMEのドキュメント節を1行更新

結果

- PDF：画像7件埋め込み確認（pypdfで検証）
- テスト、Lintは次項で再確認する

残タスク

- [x] .venvをPython 3.11で作り直す（利用者側で実施済み、3.11.15確認）

実行コマンド

```
npx.cmd -y @mermaid-js/mermaid-cli -i docs/assets/overview_ops.mmd -o docs/assets/overview_ops.png
npx.cmd -y @mermaid-js/mermaid-cli -i docs/assets/overview_internals.mmd -o docs/assets/overview_internals.png
npx md-to-pdf docs/beginner_overview.md
python -m pytest tests -q
python -m ruff check src tests config.py
```

実行成果物

- docs/assets/overview_ops.*、overview_internals.*（mmd＋png各2件）

- docs/beginner_overview.md＋.pdf（第5章追記版）
- README.md（ドキュメント節1行更新）
- docs/handover.md（本ファイル）

公開禁止情報

公開禁止情報は含まれていません

2026-09-20 review修正フェーズ（理解可否の見直し）

方針（計画）

- Python中級・RAG初心者の理解可否を再確認する
- 誤記と現状乖離を修正する。新規図は作らない

実装（実行）

- beginner_overview.md5件：略語展開、chain例context追加、構成図2行追加、誤記修正、on_event補足
- README.md7件：番号重複、動作確認節新設、Docker注記、pgvector注記2件、テスト実施済み化
- md-to-pdfでbeginner_overview.pdf再生成

結果

- テスト、Lintは次項で再確認する

残タスク

- なし

実行コマンド

```
npx md-to-pdf docs/beginner_overview.md
npx md-to-pdf docs/handover.md
python -m pytest tests -q
python -m ruff check src tests config.py
```

実行成果物

- docs/beginner_overview.md＋.pdf（review修正版）
- README.md（7件修正）
- docs/handover.md＋.pdf（本ファイル）

公開禁止情報

公開禁止情報は含まれていません

2026-09-20 Rev.A保存フェーズ（初心者文書と図の別名保存）

方針（計画）

- 初心者文書とdiagrams文書を精読し理解可否を確認する
- 原ファイルは変更せず別名Rev.A保存とする
- RAG初心者向けに読み方と実コード対応を補足する

実装（実行）

- docs/beginner_overview_RevA.md＋.pdf（です調統一、略語展開、SQLite正規明記、Docker対象外注記、簡略図注記）
- diagrams 6件のRev.A md＋mmd＋pdf（01は201/200分離、03aは依存修正、03bはRAG経路追加、04はapp.db追加、02と05は解説追加）
- READMEのドキュメント節にRev.A案内2行追加

結果

- PDF：Rev.A文書は画像7件埋め込み、図PDFは6件生成
- テスト、Lintは次項で再確認する

残タスク

- なし

実行コマンド

```
npx.cmd -y @mermaid-js/mermaid-cli -i diagrams/01_flowchart_RevA.mmd -o diagrams/01_flowchart_RevA.pdf --pdfFit
npx md-to-pdf docs/beginner_overview_RevA.md
python -m pytest tests -q
python -m ruff check src tests config.py
```

実行成果物

- docs/beginner_overview_RevA.md＋.pdf
- diagrams/*_RevA.md＋.mmd＋.pdf（6件）
- README.md（Rev.A案内）
- docs/handover.md＋.pdf（本ファイル）

公開禁止情報

公開禁止情報は含まれていません

2026-09-20 取説作成フェーズ（操作手順書）

方針（計画）

- 操作手順に特化した取扱説明書を新規作成する
- コード解説は初心者文書へ委譲し重複を避ける

実装（実行）

- docs/assets/user_manual_flow.mmd＋.png（操作フロー図、/docs表記の修正あり）
- docs/user_manual.md＋.pdf（起動、/docs読方、登録、会話、異常時、停止）
- READMEのドキュメント節に取説案内1行追加

結果

- PDF：画像埋め込み確認（pypdfで検証）
- テスト、Lintは次項で再確認する

残タスク

- なし

実行コマンド

```
npx.cmd -y @mermaid-js/mermaid-cli -i docs/assets/user_manual_flow.mmd -o docs/assets/user_manual_flow.png --pdfFit
npx md-to-pdf docs/user_manual.md
python -m pytest tests -q
python -m ruff check src tests config.py
```

実行成果物

- docs/user_manual.md＋.pdf
- docs/assets/user_manual_flow.mmd＋.png
- README.md（取説案内）
- docs/handover.md＋.pdf（本ファイル）

公開禁止情報

公開禁止情報は含まれていません

2026-09-20 ソース結合PDFフェーズ（全ソースの1冊化）

方針（計画）

- stockCheckerのgenerate_source_pdfs.pyを複製しwebRagSys用に配置する
- 対象はconfig＋src＋testsの26件。.envと.venvは対象外とする

実装（実行）

- py_source_export/generate_source_pdfs.py（先頭コメント付与）
- py_source_export/requirements.txt（fpdf2、pygments、本体とは分離）
- py_source_export/webRagSys_source.pdf（表紙・目次・本文・索引の34頁）
- READMEのドキュメント節に結合PDF案内1行追加

結果

- PDF：26件収録、表紙・目次・本文・索引を確認
- テスト、Lintは次項で再確認する

残タスク

- なし

実行コマンド

```
.¥.venv¥Scripts¥python.exe -m pip install -r py_source_export/requirements.txt
.¥.venv¥Scripts¥python.exe py_source_export/generate_source_pdfs.py . -e .py -o py_source_export/webRagSys_source.pdf --title
"webRagSys Source Code" --exclude-dir py_source_export,diagrams,docs
python -m pytest tests -q
python -m ruff check src tests config.py
```

実行成果物

- py_source_export/generate_source_pdfs.py＋requirements.txt
- py_source_export/webRagSys_source.pdf
- README.md（結合PDF案内）
- docs/handover.md＋.pdf（本ファイル）

公開禁止情報

公開禁止情報は含まれていません

2026-09-29 ローカルLLM・Postgres対応フェーズ（Ollama＋4パターン）

方針（計画）

- 会話はqwen3.5:4b、Embeddingはqwen3-embedding:0.6b（1024次元）で統一する
- config.pyのみでなくprovider／embeddings／models／requirementsを連動改修する
- DBは単一document_chunks維持、分岐毎テーブルは作らない、次元変更時は再作成＋再登録
- テストはfake/ollama × sqlite/postgresの4パターン、不可時はskipする

実装（実行）

- ollama pull qwen3-embedding:0.6b取得、/api/embedで1024次元返却確認
- requirements.txtへlangchain-ollama追加、.venvへ導入しollama-ok確認
- config.pyへllm_provider／ollama_base_url／ollama_model／ollama_embed_model／ollama_num_ctx／embedding_dim追加
- src/llm/provider.pyを3分岐化（fake／ollama ChatOllama／openai）、think=False＋num_ctx=8192

- src/rag/embeddings.pyを3分岐化（Fake8次元／OllamaEmbeddings／OpenAI）、関数内get_settings化
- src/rag/vectorstore.pyのTOP_K固定を関数内取得化、src/db/models.pyの次元を設定参照化
- src/db/database.pyのpgvector拡張失敗時にRuntimeError化、password内#は%23注記
- .env.exampleへOllama変数とEMBEDDING_DIM見本追加
- tests/test_backends.py新設（A fake+sqlite／B fake+PostgreSQL／C ollama+sqlite／D ollama+PostgreSQL。PGケースは別process E2E）
- ruff BLE001／LOG015／I001／E501修正

結果

- ruffエラー0（python -m ruff check src tests config.py → All checks passed）
- pytest 9 passed 2 skipped（既存7件＋C2件成功、B/Dはpgvector未導入のためskip）
- 実HTTP検証（port 8004、sqlite＋ollama）：POST /documents→id=1、POST /chat→「休暇は 10 日です。」＋source_ids=1、FAKEなし確認
- Postgres 18稼働確認（port 5432疎通）、psql.exe存在確認、share/extensionにvectorなし→pgvector未導入確定

残タスク

- [ ] pgvector導入（PG18 Windowsへ拡張追加、確認SQL：SELECT * FROM pg_available_extensions WHERE name='vector'）
- [ ] TEST_DATABASE_URL設定（password内#は%23符号化、例：pass%23word）
- [ ] B/D実行（Bは8次元Vector、DはEMBEDDING_DIM=1024別processで再作成＋再登録後にE2E）
- [ ] READMEとbeginner_overviewへOllama運用追記（任意）

実行コマンド

```
# 依存
.¥.venv¥Scripts¥python.exe -m pip install -q "langchain-ollama>=0.2"
# Lint／テスト
.¥.venv¥Scripts¥python.exe -m ruff check src tests config.py
.¥.venv¥Scripts¥python.exe -m pytest tests -q
# Ollama実機検証（別process起動→検証→停止）
$env:USE_FAKE="false"; $env:LLM_PROVIDER="ollama"; $env:EMBEDDING_DIM="1024"
.¥.venv¥Scripts¥python.exe -m uvicorn src.api.main:app --port 8004
# pgvector確認（psqlのフルパスで実行）
& "$env:ProgramFiles¥PostgreSQL¥18¥bin¥psql.exe" -h 127.0.0.1 -U postgres -d postgres -c "SELECT * FROM pg_available_extensions
WHERE name='vector';"
```

実行成果物

- config.py、requirements.txt、.env.example
- src/llm/provider.py、src/rag/embeddings.py、src/rag/vectorstore.py
- src/db/models.py、src/db/database.py
- tests/test_backends.py
- docs/handover.md（本ファイル）

公開禁止情報

公開禁止情報は含まれていません

2026-09-29 初心者資料Rev.E補足フェーズ（Embeddingと文書別の索引更新）

方針（計画）

- Rev.Dを変更せず複製し、Rev.Eとして補足節を追加する
- 直近の質問・回答を初心者向けに整理し、末尾に1節追加する
- 実コードの行番号と具体例を示し、製造元が変わる索引カードのたとえを含める
- 変更箇所は右側線で示し、Markdownと同名PDFを作成する

実装（実行）

- docs/beginner_overview_RevD.mdを複製してdocs/beginner_overview_RevE.mdを作成

- Rev.E末尾に第6章を追加：pgvectorの現状、EMBEDDING_DIM、チャットモデルとEmbeddingモデルの違い、think=False、文書ID単位のCRUDと索引更新
- Embeddingモデル変更時は、次元が同じなら全チャンク再Embedding、次元も変わる場合はDB列変更も必要であると区別
- ローカル設定例と2026-09-29時点のPostgreSQL 18／pgvector未導入状況を記録
- READMEのドキュメント一覧にRev.Eを追加
- docs/pdf_config.jsonを追加し、PDF下部にページ番号／総ページ数を表示
- Rev.E PDFをmd-to-pdfで生成し、ページ番号と右側線表示を確認

結果

- beginner_overview_RevD.mdは変更せず、Rev.Eを追加
- READMEにRev.E案内を追加
- beginner_overview_RevE.pdf生成済み（17ページ、ページ番号、末尾補足節、赤色右側線を確認）
- 文書のみの変更のため、pytest／ruffは再実行対象外（直前のローカルLLMフェーズでpytest 9 passed 2 skipped、ruff成功を確認済み）
- README.pdfとdocs/handover.pdfを再生成

残タスク

- なし

実行コマンド

```
# 初心者資料
npx.cmd md-to-pdf docs/beginner_overview_RevE.md --config-file docs/pdf_config.json
# README／handover PDF
npx.cmd md-to-pdf README.md --config-file docs/pdf_config.json
npx.cmd md-to-pdf docs/handover.md --config-file docs/pdf_config.json
```

実行成果物

- docs/beginner_overview_RevE.md、docs/beginner_overview_RevE.pdf
- docs/pdf_config.json
- README.md、README.pdf
- docs/handover.md、docs/handover.pdf

公開禁止情報

公開禁止情報は含まれていません

2026-09-29 README・GitHub About更新と公開情報再点検フェーズ

方針（計画）

- README冒頭の説明を従来の約2倍にし、同一文をGitHub Aboutにも設定する
- READMEの技術スタックを現行のFake／OpenAI／Ollama、Qwen、SQLite／PostgreSQL構成に合わせる
- セットアップと起動の両方にPowerShell／cmd別のvenv有効化手順を記載する
- GitHub公開中のファイルと作業ツリーを再点検し、公開禁止情報候補を一般化する
- README.pdfと本handoverのPDFをページ番号付きで再生成する

実装（実行）

- README冒頭をRAG、文書検索、Fake／OpenAI／Ollama選択を含む説明に更新し、GitHub Aboutにも同じ説明を設定
- READMEの技術スタック表を更新：SQLite、PostgreSQL＋pgvector、LangChain／langchain-ollama、Qwen3.5:4b、qwen3-embedding:0.6b
- READMEの「セットアップ」「起動」にPowerShell（Activate.ps1）とcmd（activate.bat）を別々に記載
- AGENTS.mdのWindowsパス例を一般表現へ変更
- docs/handover.mdの公開先表記からGitHub所有者ハンドルを除去
- docs/pdf_config.jsonを使いREADME.pdfとdocs/handover.pdfを再生成

公開情報監査結果

- origin/mainと作業ツリーの現行ファイルを確認し、追跡対象の.envなし、実APIキー／トークンなし

- 現行ファイルにあったWindowsパスの説明用例とGitHub所有者ハンドルを一般表現へ変更
- READMEのxxxxおよびDB接続サンプルのuser/passwordは説明用プレースホルダーであり、実認証情報ではない
- Git履歴のAGENTS.mdにWindowsパスの説明用例、旧handoverに公開リポジトリURLが残る。実パス・キー・トークンではなく、履歴改変は行わない

結果

- READMEの説明文とGitHub Aboutの説明文を一致させた
- 技術スタック表と両シェルのvenv手順を更新
- README.pdfとdocs/handover.pdfをページ番号付きで再生成
- GitHub AboutのdescriptionがREADME冒頭と一致することを確認
- 文書・GitHub Aboutのみの変更のためpytest／ruffは再実行対象外（直前のコード検証でruff成功、pytest 9 passed 2 skipped）

残タスク

- なし

実行コマンド

```
# README／handover PDF
npx.cmd md-to-pdf README.md --config-file docs/pdf_config.json
npx.cmd md-to-pdf docs/handover.md --config-file docs/pdf_config.json
# GitHub About（descriptionはREADME冒頭と同一文）
gh repo edit <owner>/<repo> --description "<README冒頭の説明文>"
```

実行成果物

- README.md、README.pdf
- AGENTS.md
- docs/handover.md、docs/handover.pdf
- GitHub repository About description

公開禁止情報

公開禁止情報は含まれていません